

My Project

Generated by Doxygen 1.12.0

1 OOP	1
1.0.1 Apie programa:	1
1.1 Kompiuterio specifikacijos:	1
1.2 Naudojimo instrukcija:	1
1.2.0.1 Naudojant kompiliatorių:	1
1.2.0.2 Naudojant cmake:	2
1.3 v2.0	2
1.4 Detalus aprašymas:	2
1.5 Testai:	3
1.6 v1.5	3
1.7 Aprašymas:	3
1.8 Detalus aprašymas:	3
1.9 v1.2	4
1.10 Aprašymas:	4
1.11 Detalus aprašymas:	5
1.11.0.1 Tai sutaupo kodo eilutes:	5
1.11.0.2 Tai sutaupo kodo eilučių tokiose vietose:	6
1.11.0.3 autotoc_md17	6
1.12 User experience:	6
1.12.0.1 Prieš:	6
1.12.0.2 Po:	6
1.12.1 Išvedimo failai:	6
1.12.1.1 Prieš:	6
1.12.1.2 Po:	6
1.13 v1.1	6
1.14 Aprašymas:	6
1.15 Repozitorijos klonavimas:	6
1.16 Spartos analizė:	6
1.16.0.1 3 strategija su vektoriai <code>struct</code> ir <code>class</code> palyginimas:	6
1.16.0.2 10 000 000:	6
1.16.0.3 1 000 000:	7
1.16.0.4 100 000:	7
1.16.0.5 Optimizavimo strategijų (O1, O2, O3) laiko palyginimas:	7
1.16.0.6 10 000 000:	7
1.16.0.7 1 000 000:	7
1.16.0.8 100 000 Įrašų Laikų Palyginimas	7
1.17 v1.0	8
1.17.0.1 Pokyčiai:	8
1.17.0.2 Strategijų aprašymas:	8
1.17.0.3 Funkcijų aprašymai:	8
1.17.0.4 Laikai	8
1.17.0.5 1000	8

1.17.0.6 10,000	9
1.17.0.7 100,000	9
1.17.0.8 1,000,000	9
1.17.0.9 10,000,000	10
1.18 v0.3	10
1.18.0.1 Atnaujinimai:	10
1.18.0.2 Funkcijų aprašymai:	10
1.18.0.3 Laikai	11
1.19 v0.2	12
1.19.0.1 Funkcijų aprašymai:	12
1.19.0.2 Laikai:	12
1.19.0.3 Veikimo schema	13
1.20 V0.1	13
1.20.0.1 Atnaujinimai:	13
1.20.0.2 Detalus atnaujinimų aprašymas:	14
2 Hierarchical Index	15
2.1 Class Hierarchy	15
3 Class Index	17
3.1 Class List	17
4 File Index	19
4.1 File List	19
5 Class Documentation	21
5.1 Stud Class Reference	21
5.1.1 Member Function Documentation	22
5.1.1.1 abstraktiFunkcija()	22
5.2 zmogus Class Reference	22
6 File Documentation	23
6.1 header.h	23
6.2 Struct.h	23
Index	25

Chapter 1

OOP

1.0.1 Apie programa:

Tai yra studentų vertinimo sistema, leidžianti apskaičiuoti studentų galutinį rezultatą pagal namų darbų ir egzamino balus.

1.1 Kompiuterio specifikacijos:

- Processor AMD Ryzen 9 6900HS with Radeon Graphics, 3301 Mhz, 3.30 GHz, 8 Core(s), 16 Logical Processor(s),
- Installed Physical Memory (RAM) 16.0 GB
- 1TB SSD storage

1.2 Naudojimo instrukcija:

1.2.0.1 Naudojant kompiliatorių:

1. Nuklonuojame repozitoriją:

```
git clone https://github.com/gurudanielius/OOP/  
cd OOP
```

1. Sukompiliuojame kodą:

```
g++ -std=c++11 *.cpp -o oop
```

1. Paleidžiame programą:

```
./oop
```

1.2.0.2 Naudojant cmake:

1. Terminale nurodome direktorija, kur egzistuoja projekto CMakeLists.txt failas, sukuriame build failams skirta direktorija :

```
cd "[direktorija]"
mkdir build
cd build
```

1. Paleidžiame cmake, kad sugeneruoti build failus, nurodome jau minėta direktorija:

```
cmake "[direktorija]"
```

1. Sukompiliuojame ir sukuriame .exe failą, taip pat galime pasirinkt konfigūracija: Release arba Debug:

```
cmake --build . --config [Release arba Debug]
```

1.3 v2.0

Aprašymas:

- Cmake pritaikytas GTest frameworkui
- Realizuoti unit testai
- Sukurta Doxygen dokumentacija

1.4 Detalus aprašymas:

Cmake:

```
cmake_minimum_required(VERSION 3.29)
project(OOP_v_pradin_)
```

```
set(CMAKE_CXX_STANDARD 14)
```

```
add_executable(Projektas
    header.h
    main.cpp
    Struct.cpp
    Struct.h)
```

```
add_executable(ProjektasTests
    testai/Tests.cpp
    testai/Structcpp_testai.cpp
    Struct.cpp)
```

```
find_package(GTest REQUIRED)
target_link_libraries(ProjektasTests GTest::gtest GTest::gtest_main)
enable_testing()
add_test(NAME ProjektasTests COMMAND ProjektasTests)
```

1.5 Testai:

Tests.cpp, čia testuojamas default konstruktorius, geteriai-seteriai, copy-constructor. Testai sėkmingi:

```
#include <gtest/gtest.h>
#include "../Struct.h"
TEST(Studentai, Konstruktorius) {
    Stud student;
    EXPECT_EQ(student.getVardas(), "petras");
    EXPECT_EQ(student.getPavarde(), "petraitis");
    EXPECT_EQ(student.rez, 10);
}

TEST(Studentai, geteriai_seteriai) {
    Stud student;
    student.setVardas("vardas");
    student.setPavarde("pavarde");
    EXPECT_EQ(student.getVardas(), "vardas");
    EXPECT_EQ(student.getPavarde(), "pavarde");
}

TEST(Studentai, CopyKonstruktorius) {
    Stud student;
    student.setVardas("vardas");
    student.setPavarde("pavarde");
    student.vid = 5;
    student.med = 6;
    student.rez = 7;
    Stud student2 = student;
    EXPECT_EQ(student2.getVardas(), "vardas");
    EXPECT_EQ(student2.getPavarde(), "pavarde");
    EXPECT_EQ(student2.vid, 5);
    EXPECT_EQ(student2.med, 6);
    EXPECT_EQ(student2.rez, 7);
}
```

1.6 v1.5

1.7 Aprašymas:

- Sukurta bazinė, abstrakti klasė žmogus, turinti atributus vardas, pavardė, šių atributų seterius, geterius.
- Virtualius metodus, kuris padaro klasę abstrakčia.
- Studento klasė dabar yra išvestinė iš Bazinės.
- Palaikoma trijų metodų taisyklė
 - Destruktorius
 - Kopijavimo konstruktorius perrašymas
 - Kopijavimo operatorius perrašymas

1.8 Detalus aprašymas:

- Klasė žmogus:


```
class žmogus {
protected:
    string vardas;
    string pavarde;
public:
    string getVardas() const { return vardas; }
    string getPavarde() const { return pavarde; }
    void setVardas(string vardas) { this->vardas = vardas; }
    void setPavarde(string pavarde) { this->pavarde = pavarde; }
    virtual void abstraktiFunkcija() const = 0;
};
```

- Klase Stud (p. 21):

```
class Stud : public zmogus {
    vector<double> ND;
    double egz;
public:
    void abstraktiFunkcija() const override {

    }
    friend std::istream& operator>>(std::istream& in, Stud& studentas) {
        std::cout << "Iveskite varda: ";
        in >> studentas.vardas;
        std::cout << "Iveskite pavarde: ";
        in >> studentas.pavarde;
        return in;
    }
    friend std::ostream& operator<<(std::ostream& out, const Stud& studentas) {
        out<<left<setw(15)<<studentas.vardas<< " "
            <<left<setw(15)<<studentas.pavarde<< " "
            <<left<setw(15)<<fixed<setprecision(2)<<studentas.med<< " "
            <<left<setw(15)<<fixed<setprecision(2)<<studentas.vid<< " "
            <<left<setw(15)<<fixed<setprecision(2)<<studentas.rez<<" "
            <<left<setw(20)<<fixed<setprecision(2)<<&&studentas<<endl;
        return out;
    }
    double vid;
    double med;
    double rez;
    ~Stud() { }
    Stud() : rez(10) { setVardas("petras"), setPavarde("petraitis"); };
    Stud(std::istream& is);
    vector<double> getND() const { return ND; }
    double getEgz() const { return egz; }
    void setND(vector<double> ND) { this->ND = ND; }
    void setEgz(double egz) { this->egz = egz; }
    Stud(const Stud& a) {
        this->vardas = a.vardas;
        this->pavarde = a.pavarde;
        this->ND = a.ND;
        this->egz = a.egz;
        this->vid = a.vid;
        this->med = a.med;
        this->egz = a.egz;
        this->rez = a.rez;
    }
    Stud& operator=(const Stud& a) {
        if (this == &a) return *this;
        this->vardas = a.vardas;
        this->pavarde = a.pavarde;
        this->ND = a.ND;
        this->egz = a.egz;
        this->vid = a.vid;
        this->med = a.med;
        this->rez = a.rez;
        return *this;
    }
};
```

Bandymas sukurti žmogaus, bet ne studento objektą išmeta klaidą:

```
zmogus Temp;
error: cannot declare variable 'Temp' to be of abstract type 'zmogus'
   96 |         zmogus Temp;
      |         ^~~~
```

Tačiau Studento sukūrimas puikiai veikia.

```
Stud zmogus;
cout<<zmogus<<endl;
```

1.9 v1.2

1.10 Aprašymas:

Realizuota Rule of three: perrašytas kopijavimo konstruktorius ir perrašytas "+" kopijavimo operatorius. Taip pat buvo perrašyti ">>" ir "<<" operatoriai.

1.11 Detalus aprašymas:

Rule of three:

- **##** Iš praeitos versijos paliktas destruktoriaus:

```
cpp
~Stud() { }
```

- **##** Perrašytas **kopijavimo operatorius**: Šis operatorius suveikia tada, kai jau egzistuojantis objektas įgyja kito objekto reikšmes. Kairėje esantis objektas įgyja lygybės dešinėje esančio objekto atributų reikšmes. Parametras **Stud** (p. 21) a yra `const` tipo ir reference type. Taip padaryta dėl to, kad jis būtų tik skaitomas ir būtų sutaupoma atmintis išvengiant nereikalingų kopijų. skiriasi nuo tradicinio kopijavimo konstruktoriaus inicializavimo, nes nebuvo naudojama dinaminė kompiuterio atmintis

```
cpp
Stud& operator=(const Stud& a) {
    if (this == &a) return *this;
    this->vardas = a.vardas;
    this->pavarde = a.pavarde;
    this->ND = a.ND;
    this->egz = a.egz;
    this->vid = a.vid;
    this->med = a.med;
    this->rez = a.rez;
    return *this;
}
```

- **##** Perrašytas ****kopijavimo konstruktorius****:

Kopijavimo konstruktorius taip pat buvo perrašytas panašiu principu, jis yra iškvičiamas kai naujai sukuriamas objektas įgyja jau egzistuojančio objekto reikšmes. Tai skiriasi nuo tradicinio kopijavimo konstruktoriaus inicializavimo, nes nebuvo naudojama dinaminė kompiuterio atmintis

cpp **Stud(const Stud& a)** (p. 21) { this->vardas = a.vardas; this->pavarde = a.pavarde; this->ND = a.ND; this->egz = a.egz; this->vid = a.vid; this->med = a.med; this->rez = a.rez; }

- **##** Perrašytas **>> operatorius**: Dabar **>> operatorius** naudojamas su **Stud** (p. 21) objektu iškart įveda vardo ir pavardės atributus.

```
friend std::istream& operator>>(std::istream& in, Stud& studentas) {
    std::cout << "Iveskite varda: " << endl;
    in >> studentas.vardas;

    std::cout << "Iveskite pavarde: " << endl;
    in >> studentas.pavarde;

    return in;
}
```

1.11.0.1 Tai sutaupo kodo eilutes:

- **##** Perrašytas **<< operatorius**: Dabar **<< operatorius** naudojamas su **Stud** (p. 21) objektu iškart įveda vardo ir pavardės atributus.

```
friend std::ostream& operator<<(std::ostream& out, const Stud& studentas) {
    out<<left<setw(15)<<studentas.vardas<< " "
        <<left<setw(15)<<studentas.pavarde<< " "
        <<left<setw(15)<<fixed<setprecision(2)<<studentas.med<< " "
        <<left<setw(15)<<fixed<setprecision(2)<<studentas.vid<< " "
        <<left<setw(15)<<fixed<setprecision(2)<<studentas.rez<< " "
        <<left<setw(20)<<fixed<setprecision(2)<<studentas<<endl;

    return out;
}
```

Dabar **<< operatorius** iškart išveda vardo, pavardės, med, vid, rez atributus, taip pat nustato kairinę lygiuotę, tarpus bei skaičiaus tikslumą ten kur reikia.

1.11.0.2 Tai sutaupo kodo eilučių tokiose vietose:

1.11.0.3 autotoc_md17

1.12 User experience:

Tiek įvedimas, tiek išvedimas vartotojui nepakito, pvz. duomenų įvedimas rankomis:

1.12.0.1 Prieš:

1.12.0.2 Po:

1.12.1 Išvedimo failai:

1.12.1.1 Prieš:

1.12.1.2 Po:

1.13 v1.1

1.14 Aprašymas:

Atsirado klasės realizacija, buvo pasirinktas vektoriaus tipo konteineris ir 3 dalinimo strategija (iš v1.0). Testuojami tik failai sudaryti iš 1 000 000 ir 10 000 000 studentų ir lyginami su atitinkama 'struct' tipo užduoties rezultatais. Taip pat išmatuotos O1, O2, O3 optimizavimo strategijų laikai.

1.15 Repozitorijos klonavimas:

1. Klonavimas:

```
git clone https://github.com/gurudanielius/OOP/
```

2. Pasikeičiam remote adresą:

```
git remote set-url origin https://github.com/gurudanielius/OOP2
```

3. Nupushiname:

```
git push -u origin v1.0
```

1.16 Spartos analizė:

Didesnius duomenų kiekius greičiausiai dalina class ir O1 optimizavimas (lenkia net ir struct), tuo tarpu mažesnius greičiausiai dalina class O2 optimizavimas (taip pat lenkia struct). Jei class nėra optimizuota, tai struct dalina greičiau tiek mažesnius, tiek didesnius duomenų kiekius, tokiu atveju struct programos laikas yra trumpesnis visose 5 matuotose srityse (apart 10 000 000 rūšiavimo laiko).

1.16.0.1 3 strategija su vektoriai struct ir class palyginimas:

1.16.0.2 10 000 000:

	Failo iš įrašų nuskaitymo laikas	Įrašų rūšiavimo laikas	Įrašų dalijimo į dvi grupes laikas	Slabakų išvedimo į failą laikas	Ramekų išvedimo į failą laikas
struct	57.789181	19.870714	3.535079	24.151542	34.317678
class	80.741019	12.709376	11.763234	34.824793	50.290692

1.16.0.3 1 000 000:

	Failo iš įrašų nuskaitymo laikas	Įrašų rūšiavimo laikas	Įrašų dalijimo į dvi grupes laikas	Slabakų išvedimo į failą laikas	Ramekų išvedimo į failą laikas
struct	3.881836	0.723616	0.194551	2.475142	3.599115
class	9.087334	0.927201	0.653186	3.534493	4.856127

1.16.0.4 100 000:

	Failo iš įrašų nuskaitymo laikas	Įrašų rūšiavimo laikas	Įrašų dalijimo į dvi grupes laikas	Slabakų išvedimo į failą laikas	Ramekų išvedimo į failą laikas
struct	0.392623	0.039085	0.037620	0.255086	0.355554
class	1.017983	0.054632	0.058039	0.513572	0.580734

1.16.0.5 Optimizavimo strategijų (O1, O2, O3) laiko palyginimas:

1.16.0.6 10 000 000:

Optimization Level	Failo iš įrašų nuskaitymo laikas	Įrašų rūšiavimo laikas	Įrašų dalijimo į dvi grupes laikas	Slabakų išvedimo į failą laikas	Ramekų išvedimo į failą laikas
O1	64.868174	10.089373	2.419835	30.585810	42.299866
O2	47.086795	9.124450	2.434581	35.550312	52.164128
O3	53.879896	8.736600	2.757521	28.816459	40.996770

1.16.0.7 1 000 000:

O	Failo iš įrašų nuskaitymo laikas	Įrašų rūšiavimo laikas	Įrašų dalijimo į dvi grupes laikas	Slabakų išvedimo į failą laikas	Ramekų išvedimo į failą laikas
O1	5.372545	0.641815	0.197577	3.983004	4.195077
O2	6.024873	0.798799	0.187038	3.381920	4.162692
O3	5.134171	0.801610	0.196136	3.864410	4.194973

1.16.0.8 100 000 Įrašų Laikų Palyginimas

Optimizacijos Lygis	Failo iš įrašų nuskaitymo laikas (s)	Įrašų rūšiavimo laikas (s)	Įrašų dalijimo į dvi grupes laikas (s)	Slabakų išvedimo į failą laikas (s)	Ramekų išvedimo į failą laikas (s)
O1	0.621154	0.025546	0.026564	0.417193	0.648291

O2	0.737948	0.024996	0.037737	0.644192	0.632701
O3	0.616188	0.026672	0.029621	0.397278	0.632775

1.17 v1.0

1.17.0.1 Pokyčiai:

Dabar pateikiamos skirtingos programos realizacijos naudojant skirtingas studentų rūšiavimo strategijas (bendro studentai konteinerio skaidymas, bendro konteinerio skaidymas panaudojant tik vieną naują konteinerį, efektyvesnės strategijos realizavimas naudojant "efektyvesius" metodus).

1.17.0.2 Strategijų aprašymas:

- 1 strategija: bendro studentai konteinerio (vector ir list tipų) skaidymas (rūšiavimas) į du naujus to paties tipo konteinerius: "slabakai" ir "ramekai".
- 2 strategija: Bendro studentų konteinerio (vector ir list) skaidymas (rūšiavimas) panaudojant tik vieną naują konteinerį: "slabakai" ir pašalinant iš pirminio konteinerio.
- 3 strategija: tai yra 2 strategija panaudojant "efektyvaus" darbo su konteineriais metodą `std::remove_if` ir listams, ir vektoriams.

1.17.0.3 Funkcijų aprašymai:

- `output2()` - sukuriamas išvedimo tekstinis failas.
- `segregacija()` - studentai yra sugrupuojami į "slabakus" ir "stiprekus", tada vartotojas pats pasirenka pagal kokį atributą (Vardas, Pavardė, Vidurkis, Mediana) šiuos studentus rušiuoti
- `DabartinisLaikas()` - grąžina dabartinį laiką
- `Skirtumas()` - grąžina dviejų laikų skirtumą
- `Generavimas()` - sugeneruojamas norimas studentų skaičius, namų darbų skaičių pasirenka vartotojas. Rezultatas yra išvedamas į tekstinį failą.

1.17.0.4 Laikai

Kiekvienas laikas kiekvienam kiekiui ir strategijai buvo matuojamas 5 kartus ir išvestas vidurkis Rūšiavimas greičiausiai veikia:

- 1000: su 3_strategija ir list.
- 10000: 3_strategija list
- 100000: 3_strategija vektoriai
- 1000000: 3_strategija vektoriai
- 10000000: remove_if vektoriai.

1.17.0.5 1000

	Failo iš įrašų nuskaitymo laikas	Įrašų rūšiavimo laikas	Įrašų dalijimo į dvi grupes laikas	Slabakų išvedimo į failą laikas	Ramekų išvedimo į failą laikas
1_strategija vektoriai	0.004083	0.000800	0.000408	0.002991	0.004089
1_strategija lis- tai	0.007082	0.000341	0.000464	0.005859	0.004645
2_strategija vektoriai	0.003850	0.000626	0.000333	0.003840	0.004104
2_strategija lis- tai	0.003809	0.000189	0.000129	0.003197	0.004416
list su remove↔ _if	0.004269	0.000740	0.000280	0.003222	0.003959
remove_if vek- toriai	0.004293	0.000202	0.000378	0.002965	0.003825

1.17.0.6 10,000

	Failo iš įrašų nuskaitymo laikas	Įrašų rūšiavimo laikas	Įrašų dalijimo į dvi grupes laikas	Slabakų išvedimo į failą laikas	Ramekų išvedimo į failą laikas
1_strategija vektoriai	0.036271	0.008440	0.003223	0.024280	0.032235
1_strategija lis- tai	0.038414	0.002534	0.003485	0.024945	0.034855
2_strategija vektoriai	0.037203	0.008531	0.002099	0.024473	0.034082
2_strategija lis- tai	0.037658	0.002442	0.001647	0.024898	0.034779
list su remove↔ _if	0.037578	0.008996	0.002608	0.028195	0.040938
remove_if vek- toriai	0.038736	0.002506	0.002670	0.024515	0.035155

1.17.0.7 100,000

	Failo iš įrašų nuskaitymo laikas	Įrašų rūšiavimo laikas	Įrašų dalijimo į dvi grupes laikas	Slabakų išvedimo į failą laikas	Ramekų išvedimo į failą laikas
1_strategija vektoriai	0.356733	0.114551	0.033200	0.228788	0.333200
1_strategija lis- tai	0.363788	0.027505	0.033392	0.240831	0.343392
2_strategija vektoriai	0.391728	0.040480	143.312432	0.249972	0.357057
2_strategija lis- tai	0.384077	0.043515	0.018041	0.255114	0.356047
list su remove↔ _if	0.392623	0.039085	0.037620	0.255086	0.355554
remove_if vek- toriai	0.388184	0.723616	0.017214	0.264142	0.351554

1.17.0.8 1,000,000

	Failo iš įrašų nuskaitymo laikas	Įrašų rūšiavimo laikas	Įrašų dalijimo į dvi grupes laikas	Slabakų išvedimo į failą laikas	Ramekų išvedimo į failą laikas
1_strategija vektoriai	3.614022	1.502412	0.325965	2.286499	3.325965
1_strategija listai	3.616999	0.450904	0.334312	2.442912	3.434312
2_strategija vektoriai	-	-	-	-	-
2_strategija listai	3.714822	0.760519	0.184551	2.461227	3.614732
list su remove↔_if	5.004823	0.796152	0.376493	3.704613	4.774052
remove_if vektoriai	3.881836	0.723616	0.194551	2.475142	3.599115

1.17.0.9 10,000,000

	Failo iš įrašų nuskaitymo laikas	Įrašų rūšiavimo laikas	Įrašų dalijimo į dvi grupes laikas	Slabakų išvedimo į failą laikas	Ramekų išvedimo į failą laikas
1_strategija vektoriai	37.966504	19.345238	9.914230	24.457310	34.917649
1_strategija listai	37.363967	7.210146	10.168519	24.865331	35.268519
2_strategija vektoriai	-	-	-	-	-
2_strategija listai	57.873212	11.589878	4.099502	25.236396	35.156699
list su remove↔_if	40.479116	10.675159	8.287767	25.221774	35.817448
remove_if vektoriai	57.789181	19.870714	3.535079	24.151542	34.317678

1.18 v0.3

1.18.0.1 Atnaujinimai:

Stud (p. 21) objektai dabar yra talpinami List konteineryje.

1.18.0.2 Funkcijų aprašymai:

- `output2()` - sukuriamas išvedimo tekstinis failas.
- `segregacija()` - studentai yra sugrupuojami į "slabakus" ir "stiprekus", tada vartotojas pats pasirenka pagal kokį atributą (Vardas, Pavardė, Vidurkis, Mediana) šiuos studentus rūšiuoti
- `DabartinisLaikas()` - grąžina dabartinį laiką
- `Skirtumas()` - grąžina dviejų laikų skirtumą
- `Generavimas()` - sugeneruojamas norimas studentų skaičius, namų darbų skaičių pasirenka vartotojas. Rezultatas yra išvedamas į tekstinį failą. #

1.18.0.3 Laikai

Buvo išmatuoti 5 tekstinių failų generavimo trukmė. Šie tekstiniai failai buvo sudaryti iš 1 000, 10 000, 100 000, 1 000 000, 10 000 000 irasu. Lentelė, vaizduojanti generavimo laiką skirtingam studentų generavimo kiekiui:

- ### 1000:

Veiksmas	Vidurkis (s)
Failo iš 1000 įrašų nuskaitymo laikas	0.007082s
1000 įrašų rūšiavimo laikas	0.000341s
1000 įrašų dalijimo į dvi grupes laikas	0.004645s
1000 slabakų išvedimo į failą laikas	0.005859s
1000 ramekų išvedimo į failą laikas	0.004645s

- ### 10000:

Veiksmas	Vidurkis (s)
Failo iš 10000 įrašų nuskaitymo laikas	0.038414s
10000 įrašų rūšiavimo laikas	0.002534s
10000 įrašų dalijimo į dvi grupes laikas	0.034855s
10000 slabakų išvedimo į failą laikas	0.024945s
10000 ramekų išvedimo į failą laikas	0.034855s

- ### 100000:

Veiksmas	Vidurkis (s)
Failo iš įrašų nuskaitymo laikas	0.363788s
Įrašų rūšiavimo laikas	0.027505s
Įrašų dalijimo į dvi grupes laikas	0.343392s

| Slabakų išvedimo į failą laikas | 0.240831s | | Ramekų išvedimo į failą laikas | 0.343392s | |

- ### 1000000:

Veiksmas	Vidurkis (s)
Failo iš įrašų nuskaitymo laikas	3.616999s
Įrašų rūšiavimo laikas	0.450904s
Įrašų dalijimo į dvi grupes laikas	3.434312s
Slabakų išvedimo į failą laikas	2.442912s
Ramekų išvedimo į failą laikas	3.434312s

- ### 10000000:

Veiksmas	Vidurkis (s)
Failo iš įrašų nuskaitymo laikas	37.363967s
Įrašų rūšiavimo laikas	7.210146s
Įrašų dalijimo į dvi grupes laikas	35.268519s
Slabakų išvedimo į failą laikas	24.865331s
Ramekų išvedimo į failą laikas	35.268519s

1.19 v0.2

1. Pridėta generavimo funkcija
2. Atsirado funkcija, dalinanti studentus į dvi kategorijas. Studentai išvedami į du naujus failus:
 - Studentai, kurių galutinis balas < 5.0 - slabakai
 - Studentai, kurių galutinis balas ≥ 5.0 - ramekai
3. Pridėta funkcija, matuojanti kitų funkcijų laiką, išvedanti sugrupuotus studentus į naujus failus.
4. Nuo praeitos versijos atliktas refactoringas, dabar main.cpp veikia while-do principu. #

1.19.0.1 Funkcijų aprašymai:

- `output2()` - sukuriamas išvedimo tekstinis failas.
- `segregacija()` - studentai yra sugrupuojami į "slabakus" ir "stiprekus", tada vartotojas pats pasirenka pagal kokį atributą (Vardas, Pavarde, Vidurkis, Mediana) šiuos studentus rušiuoti
- `DabartinisLaikas()` - grąžina dabartinį laiką
- `Skirtumas()` - grąžina dviejų laikų skirtumą
- `Generavimas()` - sugeneruojamas norimas studentų skaičius, namų darbų skaičių pasirenka vartotojas. Rezultatas yra išvedamas į tekstinį failą. #

1.19.0.2 Laikai:

Buvo išmatuoti 5 tekstinių failų generavimo trukmė. Šie tekstiniai failai buvo sudaryti iš 1 000, 10 000, 100 000, 1 000 000, 10 000 000 įrasu. Lentelė, vaizduojanti generavimo laiką skirtingam studentų generavimo kiekiui:

Generavimo kiekis	Generavimo laikas
1000	0.0043209s
10000	0.0381509s
100000	0.389532s
1000000	3.7939s
10000000	39.7852s

Tada buvo matuojamas šių tekstinių failų nuskaitymo laikas, rušiavimo laikas, dalijimo į dvi grupes laikas, slabaku išvedimo į failą laikas, rameku išvedimo į failą laikas. Šiuos požymius, naudojant kiekvieną sugeneruotą failą, matavome po 5 kartus ir apskaičiavome laiko vidurkį.

- ### 1000:

Veiksmas	Vidurkis (s)
1000 įrašų rūšiavimo laikas	0.000800s
1000 ramekų išvedimo į failą laikas	0.004089s
1000 įrašų dalijimo į dvi grupes laikas	0.004089s
1000 slabakų išvedimo į failą laikas	0.002991s
Failo iš 1000 įrašų nuskaitymo laikas	0.004083s

#

- ### 10000:

Veiksmas	Vidurkis (s)
Failo iš 10000 įrašų nuskaitymo laikas	0.036271s
10000 įrašų rūšiavimo laikas	0.008440s
10000 įrašų dalijimo į dvi grupes laikas	0.032235s
10000 slabakų išvedimo į failą laikas	0.024280s
10000 ramekų išvedimo į failą laikas	0.032235s

#

- ### 100000:

Veiksmas	Vidurkis (s)
Failo iš 100000 įrašų nuskaitymo laikas	0.356733s
100000 įrašų rūšiavimo laikas	0.114551s
100000 įrašų dalijimo į dvi grupes laikas	0.333200s
100000 slabakų išvedimo į failą laikas	0.228788s
100000 ramekų išvedimo į failą laikas	0.333200s

#

- ### 1 000 000:

Veiksmas	Vidurkis (s)
Failo iš 1000000 įrašų nuskaitymo laikas	3.614022s
1000000 įrašų rūšiavimo laikas	1.502412s
1000000 įrašų dalijimo į dvi grupes laikas	3.325965s
1000000 slabakų išvedimo į failą laikas	2.286499s
1000000 ramekų išvedimo į failą laikas	3.325965s

#

- ### 10 000 000:

Veiksmas	Vidurkis (s)
Failo iš 10000000 įrašų nuskaitymo laikas	37.966504s
10000000 įrašų rūšiavimo laikas	19.345238s
10000000 įrašų dalijimo į dvi grupes laikas	34.917649s
10000000 slabakų išvedimo į failą laikas	24.457310s
10000000 ramekų išvedimo į failą laikas	34.917649s

#

1.19.0.3 Veikimo schema

1.20 V0.1

1.20.0.1 Atnaujinimai:

1. Pridėtas klaidų tvarkymas ir apribojimai šiems klausimams, skirtiems vartotojui:

- "Ar skaityti duomenis is failo? taip/ne"
- "Iveskite studentu skaiciu"
- "Ar norite ivesti pazymius rankiniu budu, ar generuoti atsitiktinai? (Ivesti/Generuoti) "
- "Iveskite pazymiu skaiciu: "
- "Iveskite namu darbu rezultatus noredami uzbaigti iveskite -1: "
- Prideta klaida, jei įvedamas neegzistuojantis arba neprieinamas nuskaitomo failo pavadinimas
- Pazymiai turi būti įvedami skaičiaus formatu ir priklausyti intervalui [1;10]

2. Rezultatas išvedimas:

- Pataisytas matmuo, kuris rodomas išvedant rezultatą.
- Rezultatas išvedamas rūšiuojant studentus pagal vardą
- Rezultatas išvedamas sulgyjuotas palei kairę pusę
- Prideta galimybė išvesti rezultatą į failą

3. Klaidų pataisymas: -Jei vartotojas įveda 0 namų darbų, nebelieka dalybos iš 0

1.20.0.2 Detalus atnaujinimų aprašymas:

- Klausimuose, kuriuose atlikti apribojimai ir klaidų tvarkymas, vartotojas yra priverstas atsakyti į klausimus. Kol atsakymo programa nepriims, tol klausimas vartotojui bus kartojamas.
- Namų darbų įvedimo pabaigimas buvo atnaujintas iš neskaitinių simbolių į skaičių "-1". Taip pat dabar programa pateikia rezultatą ir nepateikus nei vieno namų darbo.
- Namų darbų ir egzamino balai reikalauja, kad priklausytų intervalui [1;10]
- Rezultatas pateikia ne egzamino balą, o yra apskaičiuojamas naudojant namų darbų ir egzamino balus. Formulė $0.4 [ND_balas] + 0.6 [Egz.balas]$
- Vartotojo yra paklausama, ar norima rezultatą išvesti faile ar terminale. Pirmu atveju, rezultatas yra išsaugomas "rezultatas.txt", o antru rezultatas yra išvedamas ekrane. Rezultatas, tiek tekstiname faile, tiek ' terminale yra rūšiuojamas pagal vardą ir yra pritaikyta kairinė lygiuotė.
- Pridėtas try-catch blokas, kai nerandamas arba neįrašomas taisyklingas failo pavadinimas. Programa išmeta klaidą "terminate called after throwing an instance of 'std::ios_base::failure[abi:cxx11]' what(): Nepavyko atidaryti failo: iostream error <h2>Programos veikimo schema</h2> <p align="center"> </p> @section autotoc_md57 V.pradinė @subsubsection autotoc_md58 Sukurta pradinė vertinimo programos versija: Programinis kodas: 1. Sukurta struktūra, kuri aprašo studentą, atributus ir metodus. 2. Sukurti skirtingi duomenų įvedimo būdai: generavimas, įvedimas ranka nežinant namų darbų skaičiaus, nuskaitymas 3. Sukurti metodai, kurie leidžia įvesti duomenis, apskaičiuoja vidurkį ir medianą, išveda atsakymą, "atlaisvina" objektą. @subsubsection autotoc_md59 Veikimo schema ir detalus aprašymas; 1. Vartotojo yra paklausama, koku matu (vidurkiu - V ar mediana -M) norima remtis skaičiuojant galutinį rezultatą. Jei vartotojas įveda bet kokį kitą simbolį apart "V" ar "M" , tai bus skaičiuojama mediana. 2. Vartotojo yra paklausama, ar norima duomenis nuskaityti iš failo. Jei atsakymas yra taip, skaitoma iš failo "kursiokai.txt". Jei atsakymas yra "ne", tada yra paprašoma įvesti studentų skaičių. Įvedus studentų skaičių vartotojo paklausama, ar norėtų susivesti duomenis ranka, ar norėtų, kad jam juos sugeneruotų kompiuteris. Vartotojo paprašoma įvesti studento vardą ir pavardę. - Jei pasirenkama įvesti duomenis ranka, tada vartotojas veda tiek pažymių kiek nori, o pažymių įvedimą pabaigia parašydamas neskaitinį simbolį. Vartotojo galiausiai paprašoma įvesti egzamino rezultatą. - Jei pasirenkama sugeneruoti duomenis, tada paklausama, kiek namų darbų pažymių norima sugeneruoti. Sugeneruojamas ir egz. rez 3. Išvedamas rezultatas tokia struktūra: Vardas Pavardė Galutinis (← Vid.)(Med.) Galutinis rezultatas rodo tik vidurkį arba medianą priklausomai nuo vartotojo pasirinkimo. <h2>Programos veikimo schema</h2> <p align="center">

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

zmogus	22
Stud	21

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Stud	21
zmogus	22

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

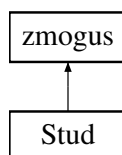
header.h		23
Struct.h		23

Chapter 5

Class Documentation

5.1 Stud Class Reference

Inheritance diagram for Stud:



Public Member Functions

- void **abstraktiFunkcija** () const override
- **Stud** (string vardas, string pavarde, vector< double > ND, double egz)
- **Stud** (std::istream &is)
- vector< double > **getND** () const
- double **getEgz** () const
- void **setND** (vector< double > ND)
- void **setEgz** (double egz)
- **Stud** (const **Stud** &a)
- **Stud** & **operator=** (const **Stud** &a)

Public Member Functions inherited from zmogus

- string **getVardas** () const
- string **getPavarde** () const
- void **setVardas** (string vardas)
- void **setPavarde** (string pavarde)

Public Attributes

- double **vid**
- double **med**
- double **rez**

Friends

- `std::istream & operator>> (std::istream &in, Stud &studentas)`
- `std::ostream & operator<< (std::ostream &out, const Stud &studentas)`

Additional Inherited Members

Protected Attributes inherited from **zmogus**

- string **vardas**
- string **pavarde**

5.1.1 Member Function Documentation

5.1.1.1 **abstraktiFunkcija()**

```
void Stud::abstraktiFunkcija () const [inline], [override], [virtual]
```

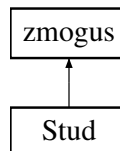
Implements **zmogus** (p. 22).

The documentation for this class was generated from the following file:

- Struct.h

5.2 **zmogus** Class Reference

Inheritance diagram for **zmogus**:



Public Member Functions

- string **getVardas** () const
- string **getPavarde** () const
- void **setVardas** (string vardas)
- void **setPavarde** (string pavarde)
- virtual void **abstraktiFunkcija** () const =0

Protected Attributes

- string **vardas**
- string **pavarde**

The documentation for this class was generated from the following file:

- Struct.h

Chapter 6

File Documentation

6.1 header.h

```
00001 //
00002 // Created by danie on 13/09/2024.
00003 //
00004 #include <random>
00005 #include <fstream>
00006 #include <iostream>
00007 #include <exception>
00008 #include <sstream>
00009 #include <list>
00010 using std::istringstream;
00011 using std::endl;
00012 extern std::string choice;
00013 extern int lines_num;
00014
00015 extern double nuskaitymo_failo_t, generavimo_failo_t, segregacija_t, sort_t, slabakai_output_t,
    ramiakai_output_t;
00016 using std::cout;
00017 using std::left;
00018 using std::cin;
00019 using std::string;
00020 using std::vector;
00021 using std::list;
00022
00023 #ifndef HEADER_H
00024 #define HEADER_H
00025 #include <iomanip>
00026 using std::setw;
00027 using std::fixed;
00028 using std::setprecision;
00029 #include <algorithm>
00030 #include <chrono>
00031
00032 #endif //HEADER_H
```

6.2 Struct.h

```
00001 #include "header.h"
00002
00003 class zmogus {
00004     protected:
00005         string vardas;
00006         string pavarde;
00007     public:
00008         virtual ~zmogus() = default;
00009
00010         string getVardas() const { return vardas; }
00011         string getPavarde() const { return pavarde; }
00012         void setVardas(string vardas) { this->vardas = vardas; }
00013         void setPavarde(string pavarde) { this->pavarde = pavarde; }
00014         virtual void abstraktiFunkcija() const = 0;
00015 };
00016 class Stud : public zmogus {
00017     vector<double> ND;
00018     double egz;
```

```

00019 public:
00020     void abstraktiFunkcija() const override {
00021     }
00022 }
00023 friend std::istream& operator>>(std::istream& in, Stud& studentas) {
00024     std::cout << "Iveskite vardas: ";
00025     in >> studentas.vardas;
00026     std::cout << "Iveskite pavarde: ";
00027     in >> studentas.pavarde;
00028     return in;
00029 }
00030 friend std::ostream& operator<<(std::ostream& out, const Stud& studentas) {
00031     out<<left<setw(15)<<studentas.vardas<< " "
00032         <<left<setw(15)<<studentas.pavarde<< " "
00033         <<left<setw(15)<<fixed<setprecision(2)<<studentas.med<< " "
00034         <<left<setw(15)<<fixed<setprecision(2)<<studentas.vid<< " "
00035         <<left<setw(15)<<fixed<setprecision(2)<<studentas.rez<<" "
00036         <<left<setw(20)<<fixed<setprecision(2)<<studentas<<endl;
00037     return out;
00038 }
00039 double vid;
00040 double med;
00041 double rez;
00042 ~Stud() { }
00043 Stud() : rez(10) { setVardas("petras"), setPavarde("petraitis"); };
00044 Stud(string vardas, string pavarde, vector<double> ND, double egz) : ND(ND), egz(egz)
{setVardas(vardas), setPavarde(pavarde);}
00045 Stud(std::istream& is);
00046 vector<double> getND() const { return ND; }
00047 double getEgz() const { return egz; }
00048 void setND(vector<double> ND) { this->ND = ND; }
00049 void setEgz(double egz) { this->egz = egz; }
00050 Stud(const Stud& a) {
00051     this->vardas = a.vardas;
00052     this->pavarde = a.pavarde;
00053     this->ND = a.ND;
00054     this->egz = a.egz;
00055     this->vid = a.vid;
00056     this->med = a.med;
00057     this->egz = a.egz;
00058     this->rez = a.rez;
00059 }
00060 Stud& operator=(const Stud& a) {
00061     if (this == &a) return *this;
00062     this->vardas = a.vardas;
00063     this->pavarde = a.pavarde;
00064     this->ND = a.ND;
00065     this->egz = a.egz;
00066     this->vid = a.vid;
00067     this->med = a.med;
00068     this->rez = a.rez;
00069     return *this;
00070 }
00071 };
00072 void ived(Stud & Lok);
00073 void vidurkis(Stud & Lok);
00074 void output(list<Stud> & Lok);
00075 void val(Stud & Lok);
00076 void mediana(Stud & Lok);
00077 void output2(list<Stud> Lok, const string& file_name);
00078 void generavimas(int n, string failo_pavadinimasm, int number_of_nd);
00079 void segregacija(list<Stud> &Lok);
00080 std::chrono::steady_clock::time_point DabartinisLaikas();
00081 double Skirtumas(std::chrono::steady_clock::time_point pradzia,
std::chrono::steady_clock::time_point pabaiga);
00082 list<Stud> nuskaitymas_is_failo(string file_name);
00083 void output_times(int lines_num, double nuskaitymo_failo_t, double sort_t, double segregacija_t,
double slabakai_output_t, double ramiakai_output_t);
00084
00085 extern std::string choice;
00086 extern int lines_num;
00087 extern double segregacija_t;
00088 extern double sort_t;
00089 extern double slabakai_output_t;
00090 extern double ramiakai_output_t;

```

Index

abstraktiFunkcija
Stud, 22

OOP, 1

Stud, 21
abstraktiFunkcija, 22

zmogus, 22